tex.web transformed to tex.cs

J. Daniel Smith

Abstract

tex.web transformed into C#; not another port, rather the WEB is preserved in IDE-friendly C#.

1 Introduction

I've transformed tex.web into C# (and C++). Note the wording; this is not another port of TEX to *some language*. Rather, tex.cs follows the presentation of the WEB while still being ordinary IDE-friendly C#. The result is a version of TEX that is structurally faithful to tex.web, usable in a modern IDE, and passes the TRIP test—without using tangle.

2 Why and why now?

The 'why now?' is easy: Knuth was born in 1938; regrettably, that means we're likely getting close to TEX version π [3]. TEX is extremely stable, tex.web isn't going to change much; so its transformation into C# (or C++) can be done manually once without fear of a moving target. Should we be fortunate enough to get TEX 3.1415926539 (currently scheduled for 2029 [1]) any changes to tex.web can readily be incorporated 'by hand.'

'Why?' is more multi-faceted. In no particular order, reasons could include:

- tex.web might be one of the more important programming artifacts; but its form makes it increasingly inaccessible to developers.
- Demonstrating that a 40-year-old Pascal program can be modernized; perhaps software engineers want to rewrite a bit too much?
- There are few non-trivial programs (other than METAFONT) which have been published as a book; it's quite unusual to have such a canonical reference for a program.
- Knuth once remarked that he was slightly disappointed in the lack of experimentation with TEX itself [5]; this, despite clear hooks and suggestions in the code itself (and even Knuth's own TEX-XET [2]).
- While function-call overhead is still important, such decisions are usually better left to the compiler rather than the programmer; especially for a program like TEX which is unlikely to be on a critical hot path.
- Programming languages now have constructs such as lambdas and reflection which make it possible to do tangle-like code manipulation in the language itself.
- The TRIP test helps ensure that the result is truly TEX, not just something that 'compiles and runs.'

3 Creating tex.cs

What follows is a description of the process of creating a readable tex.cs from tex.web. No tools were created to enable the process to be repeated; there likely isn't much interest in converting other TEXware to C# in this manner. (This isn't quite true; I first transformed DVIType, both as a proof-of-concept and for eventual use in running the TRIP test.)

Undoubtedly, the single most important tool used was Martin Ruckert's web2w [9]. This removed the tedium of munging Pascal syntax into C; it also largely implicitly answered the question of how much WEB/Pascal syntax should be preserved. For example, using the C-preprocessor, help2("a")("b") preserved while C# all but requires help2("a", "b"). web2w also removes the TEX.POOL file [10], even C has better support for strings than Standard Pascal did in 1982. Thus, much of the C# code is simply the C code from tex.w rather than hand-ported Pascal.

The next step was simple: all WEB modules were turned into comment blocks, and I added back the original Pascal for easy comparison with the C# code. To enable possible future weave-like processing, each comment block begins with /*[tex.web] and ends with [tex.web]*/.

```
/*[tex.web]
@* \[1] Introduction.
This is \TeX, a document compiler ...
[tex.web]*/
```

(This module has an empty Pascal part.) No further attempt was made to preserve any other weave features; in particular, cross-reference and index tags in code were removed or commented-out as they are invalid code; IDEs have a host of code spelunking features.

For example, at line 948 of tex.web the '@!' before bypass_eoln was just removed from the web2w transformed code:

```
boolean
input_ln(alpha_file f, boolean bypass_eoln)
```

Much of the C code produced by web2w can be compiled as C# with minimal effort; that's because tex.w is a straight-forward transformation of the Pascal code, it doesn't use a lot of C features. We get our first array at line 588, using an ArrayOf utility class, the C# is

```
/*[tex.web]
@ The {\TeX} processor converts between ...
```

```
@<Glob...@>=
@!xord: array [text_char] of ASCII_code;
 {specifies conversion of input characters}
[tex.web]*/

ArrayOf<ASCII_code> xord = new(text_char);
    // specifies conversion of input characters
```

which also nicely corresponds to the original `WEB` code.

Implementing a C-runtime in C# targeted at the specific needs of `tex.w` made it possible to have the C# version passing the TRIP test with minimal changes to `tex.w` (i.e., using `ctangle` to produce `tex.c` and then renaming that file to `tex.cs`).

`ArrayOf` is one of several pieces of a minimal Pascal runtime intended to preserve the original `WEB` code rather than convert to language-specific routines; e.g., Pascal's `reset()` instead of .NET's `File.Open()` or C's `fopen()`.

4 Preserving the structure of `tex.web`

At this point, we have 'just' a port of TEX to C#; the 'C' code in `tex.w` is now actually C#. Yes, this is interesting and useful (is there an actual TEX implementation in .NET?), but with `ctangle` still in the picture it is cumbersome to use in an IDE that is expecting real C# source files.

4.1 One large class

The key insight at this point is that both C# and C++ (as well as several other languages) allow most class members to appear in *any order*, a member can be used before it is declared. By putting all the code into a single class, the effect of `tangle`'s code-shuffling is achieved by the compiler. Doing something similar in C requires forward declarations which detracts from the `WEB` presentation. The result is that `@<Glob...@>=` (line 460) and `@<Types...@>=` (line 550) just become in-place code:

```
/*[tex.web]
@ In case somebody has inadvertently made ...

@<Glob...@>=
@!bad:integer; {is some ‘‘constant’’ wrong?}
[tex.web]*/

integer bad; /*is some ‘‘constant’’ wrong?*/
```

and

```
/*[tex.web]
@ Characters of text that have been ...

@<Types...@>=
@!ASCII_code=0..255; {eight-bit numbers}
[tex.web]*/

static readonly Range ASCII_code = 0..255;
 // eight-bit numbers
```

After that, many of the 'C' parts can become member functions as they are standalone pieces of code which only use 'global' variables (which is now member data). The named 'C' part becomes the name of a member function, replacing spaces with underscores and removing other characters that can't be part of an identifier. Yes, this becomes a bit unwieldly in places because `WEB` treats the names as `weave` text with full formatting and potentially long names.

```
/*[tex.web]
@ We need a special routine to read the ...

@<Report overflow of the input buffer,
 and abort@>=
if format_ident=0 then ...
[tex.web]*/

void
Report_overflow_of_the_input_buffer_and_abort()
{
 if (format_ident is null) ...
}
```

4.2 Local functions in C#

The first real complication is when a procedure declares some local variables which are then used in multiple subsequent parts. In C# this is handled with local functions [6]. See `short_display` at line 3541:

```
/*[tex.web]
@ Boxes, rules, inserts, whatsits, marks, ...
@p procedure short_display(@!p:integer);
 {prints highlights of list |p|}
    [...]
 @<Print a short indication of the contents
  of node |p|@>;
    [...]
[tex.web]*/

void short_display(integer p)
 /*prints highlights of list |p|*/
{
   [...]
   Print_a_short_indication_of_the__...();

   /*[tex.web]
   @ @<Print a short indication of the contents
    of node |p|@>=
   [...]
   [tex.web]*/

   void
   Print_a_short_indication_of_the_...()
   {
```

```
      [...]
 }
} /*short_display()*/
```

Notice that `short_display()` ends at the second `/*[tex.web]...[tex.web]*/` block, after the local function `Print_...()` is defined.

4.3 Reflection and `[Module]`

Some `WEB` modules are spread throughout `tex.web`, `tangle` combines these into one block of code; initialization is one example. Using reflection in C# (lambdas in C++), we can arrange for these disparate blocks of code to be called all at once (`[Module]` is a custom attribute [8]):

```
/*[tex.web]
[...]
@<Set init...@>=
for i:=0 to @'37 do xchr[i]:=' ';
[tex.web]*/

[Module("Set init...")]
void Set_initial_values_of_key_variables_2_2()
{
 for (i = 0; i <= 31; i++) xchr[i] = ' ';
}
```

Initialization often takes place immediately after declaration. Both C# and C++ (as well as other languages) allow ('prefer', and sometimes even 'require') initialization to occur at the same time as declaration. (In C# it is customary to omit initialization that is identical to default initialization).

```
/*[tex.web]
@<Glob...@>=
@!deletions_allowed:boolean;
 {is it safe for |error| to call |get_token|?}
[...]
@<Set init...@>=
deletions_allowed:=true; ...
[tex.web]*/

boolean deletions_allowed = true;
 // is it safe for |error| to call |get_token|?
[...]
```

4.4 Non-local `goto`s

Another complication is that many `WEB` modules contain `goto`s. When `tangle` combines the modules, they are local `goto`s; however, as separate functions this is no longer the case. Exceptions can be used to emulate these non-local `goto`s. Although this sounds bad, it is normally rather structured jumping, usually to a spot near the beginning or end of a significant block.

The code for catching exceptions is verbose compared to 'nothing' in `WEB`, but thankfully is rather limited. Here is the code at line 1896:

```
/*[tex.web]
@ @<Get user's advice...@>=
loop@+begin continue: [...]
  end
[tex.web]*/

void Get_users_advice()
{
 while (true)
 {
 @continue: try
  {
   [...]
  }
  catch (GotoException ex) when
   (ex.Label is "continue") { goto @continue; }
 }
}
```

4.5 Partial classes in C#

There are a few functions/procedures in `tex.web` that are too large to be presented all at once, some of them are even spread across multiple starred modules. Because of other intervening code, C#'s local functions won't work.

The solution is to use a feature unique to C#: `partial` classes [7]. This allows the contents of the class to be multiple places in the source code, each writing `partial class` *`name`*, the C# compiler combines these pieces into a single class. That, together with a helper `Ref` class to reference the same member data as the containing `TEX` class, make it possible to write the following for the code at line 12319:

```
/*[tex.web]
@ The recursive procedures |hlist_out| ...

procedure hlist_out;
 {output an |hlist_node| box}
label reswitch, move_past, fin_rule, next_p;
var base_line: scaled;
 {the baseline coordinate for this box}
[...]
begin cur_g:=0; ...
[tex.web]*/

void hlist_out() {
 new hlist_out_(this).Invoke(); }
sealed partial class hlist_out_(TEX tex)
 : Ref(tex) // output an |hlist_node| box
{
 scaled base_line;
  /*the baseline coordinate for this box*/
 [...]
```

```
public void Invoke()
{
 cur_g = 0; ...
```

The implementation of `hlist_out` continues a bit later at line 12438 after a few `WEB` macros.

```
/*[tex.web]
@ @d billion==float_constant(1000000000)
@d vet_glue(#)== ...

@<Move right or output leaders@>=
begin [...]
[tex.web]*/

static readonly double billion =
 float_constant(1_000_000_000);
void vet_glue(double A, out double ...

partial class hlist_out_
{
 void Move_right_or_output_leaders(pointer p)
 { [...]
```

4.6 Miscellaneous

As with almost any non-trivial piece of software, reality isn't quite that simple. There are a large number of minor changes needed to actually get the C# code compiling; for example, several routines in §54 *System-dependent changes* make strings (and TEX's string pool) more natural in C#.

5 What's next?

Can `tex.web` be transformed so that it is directly usable in a modern IDE? Apparently yes, even if the result isn't a good example of idiomatic C# and nobody would write a function name like `Make_...()` (very long name truncated). Nevertheless, the result remains faithful to `tex.web` while remaining ordinary C# source code.

It's almost certain that if Knuth were to write TEX today from scratch in C# (or C++), he would use the dynamic memory facilities provided by the language, i.e., `new()`, as well as many other standardized features; the result would look nothing like `tex.cs`. (Although his more recent work on MMIX for *The Art of Computer Programming* shows he still has a penchant for Literate Programming [4], so he might never give up `CWEB`).

A similar approach could probably be used for many other languages that support classes (arbitrary ordering of member names and implicit `this`) and exceptions (for non-local `goto`s).

In any case, whatever would be gained by not using 1982 Pascal would surely be lost to PDF, Unicode, OpenType and numerous other features that are part of today's robust TEX implementations. Yet, having a canonical TEX available in a modern development environment could enable experimentation, education and who knows what else?

6 Try it out for yourself

Experience the thrill of running TEX directly in the debugger (or browser), no `#line` directives needed! First, download *Visual Studio* from `visualstudio.microsoft.com/downloads/`. Then, either C# or C++ source code from `github.com/TeX-82`.

References

[1] D. Knuth. The TEX tuneup of 2021. *TUGboat* 42(1):7–10, 2021. `doi.org/10.47397/tb/42-1/tb130knuth-tuneup21`

[2] D. Knuth, P. MacKay. Mixing right-to-left texts with left-to-right texts. *TUGboat* 8(1):14–25, Apr. 1987. `tug.org/TUGboat/tb08-1/tb17knutmix.pdf`

[3] D.E. Knuth. The future of TEX and METAFONT. *TUGboat* 11(4):489–489, Nov. 1990. `tug.org/TUGboat/tb11-4/tb30knut.pdf`

[4] D.E. Knuth. *Literate Programming*. Leeland Stanford Junior University, 1992.

[5] D.E. Knuth. Questions and answers, i. In *Digital Typography*, ch. 31, pp. 597–598. Leeland Stanford Junior University, 1999.

[6] Microsoft. Local functions (c# programming guide). `learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/local-functions`, 2025. Accessed: 2026-09-24.

[7] Microsoft. Partial classes and members (c# programming guide). `learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/partial-classes-and-methods`, 2025. Accessed: 2026-09-24.

[8] Microsoft. Writing custom attributes. `learn.microsoft.com/en-us/dotnet/standard/attributes/writing-custom-attributes`, 2025. Accessed: 2026-09-24.

[9] M. Ruckert. Converting TEX from WEB to cweb. *TUGboat* 38(3):353–358, 2017. `tug.org/TUGboat/tb38-3/tb120ruckert.pdf`

[10] M. Ruckert. The WEB to CWEB conversion of TEX. *TUGboat* 42(2):139–142, 2021. `doi.org/10.47397/tb/42-2/tb131ruckert-cweb`

◇ J. Daniel Smith
3588 Plymouth Rd. #184
Ann Arbor, Michigan 48105
USA
`dan (at) jdanielsmith dot me`